

Kapitel 24. Standard Template Library

Innehållsförteckning

[Vad är STL](#)[Containers](#)[Iteratorer](#)[Generiska algoritmer](#)[Adaptorer](#)[Diskussion](#)

Detta kapitel behandlar en annan ny del av standarden för C++, nämligen *Standard Template Library*, som namnet säger är ett bibliotek med standardiserade template-klasser.

Vad är STL

STL är en förkortning av *Standard Template Library*, men är lika känt under namnet *STL*, så förkortningen används här.

Som redan nämndes i kapitlet om typparametrisering (se [Kapitel 23](#)) är templates mycket praktiska då det gäller att skapa generiska containers och klasser som kan operera på många olika datatyper. De flesta större program behöver någon form av lagring för diverse former av data och templatiserade containers (classer skapade för att "innehålla" andra klasser) är mycket praktiska. För de flesta större program skapades alltså egna container-klasser, som kanske återanvändes i andra projekt, eller så skapades nya för nästa projekt. Slutresultatet var att man "uppfann hjulet gång på gång", och alltid skapade nytt som aldrig fick en chans att bli totalt buggfritt och använt i stora kretsar. Kommiten som standardiserar C++ märkte detta och tog därför med ett bibliotek med olika template-klasser i standarden för C++. Dessa klasser är menade att skall fungera som bas för de mesta av container-behov ett program kan ha. Så föddes STL, som numera finns med i varenda modern C++-kompilatorn.

Tanken med STL är att om det finns ett standardiserat bibliotek så undviker man att uppfinna hjulet varje gång, och kan istället koncentrera sig på att programmera själva logiken i ett program. STL har även fått en chans att ordentligt stabiliseras och bli både buggfritt och väl optimerat. Man har velat göra klasserna i STL snabba och minnessnåla så att program inte "bestraffas" för att de använder sig av klasser ur STL.

Detta material kan endast skrapa på ytan på den funktionalitet som finns i STL, och biblioteket skulle egentligen kräva ett helt eget kompendium för att ge en grundläggande översikt. Några metoder för vissa klasser visas, men läsaren hänvisas till de extensiva referenshemsidor som nämns i kapitlet [Referenser](#).

Grundblock i STL

STL är ett stort bibliotek men många klasser och ännu flera användningsområden. De klasser som finns i STL kan grovt indelas i några stora huvudgrupper:

- *container-klasser* som används för att hålla ordning på objekt. Här finns bl.a. listor, köer

stackar och vektorer.

- *generiska algoritmer* som är algoritmer som opererar på containers. Dessa algoritmer fungerar på samma sätt oberoende av vilken container de används på. Exempelvis sortering är en generisk algoritm.
- *iteratorer* som används för att iterera igenom de objekt som finns i en container. Alla containers ha egna iteratorer, men deras gränssnitt ser lika ut, så användaren kan således enkelt iterera genom objekten i vilken container som helst.
- *funktionsobjekt* som fungerar som funktioner som appliceras på en container. Dessa kan utföra någon viss operation på varje element, t.ex. beräkna ett medeltal eller summor.
- *adaptorer* som adapterar gränssnittet som någon klass har för att passa andra behov, t.ex. baklängesiteratorer.

Förutom dessa finns ännu olika former av allokatörer som används för att allokera minne för de olika klassernas interna bruk. Dessa behöver man sällan befatta sig med. De flesta olika klasserna av dessa kategorier kan ganska fritt "pluggas ihop" med varandra utan skillnad. Detta är styrkan bakom generisk programmering och STL. I de följande avsnitten behandlas de olika grundblocken lite mera ingående.

Effektivitet

Alla operationer som sker med containers och funktioner i STL har optimerats för maximal effektivitet. I mera ingående dokumentation redogörs exakt för den tid det tar att accessera, radera och sätta in element i en container, samt tiden det tar att utföra olika funktioner. Dessa ges med den normala *O-notationen* (se t.ex. kursen i datastrukturer). För de flesta operatione finns även för- och eftervillkor definierade, d.v.s. vad som finns före en operation utförs, och vilket "läge" containern befinner sig i efter operationen (se t.ex. kursen i programmeringsmetodik). Denna effektivitetsinformation kommer inte att tas upp i detta material, utan det är information för en kurs i datastrukturer eller STL-programmering.

[Förutgående](#)
Multipla parametrar

[Hem](#)

[Nästa](#)
Containers